

Chapitre III

Implementation — GED- ISIPA

Audit de conformite entre le memoire
et l'application reellement deployee

Ce rapport identifie 12 ecarts majeurs et 18 ecarts mineurs
entre les descriptions du Chapitre 3 du memoire et le
code reellement deploye. Le taux de completude estime
est de 45%. Le chapitre necessite des revisions majeures
avant la soutenance.

Master en Informatique de Gestion

Institut Superieur Informatique Programmation & Administration

Annee academique 2024-2025

TABLE DES MATIERES

1. SYNTHESE EXECUTIVE	3
2. ECARTS CRITIQUES	3
2.1 Systeme RBAC decrit vs reel	3
2.2 Architecture mono-tenant vs multi-tenant SaaS	4
2.3 bcrypt non implemente	4
2.4 Donnees de seed incorrectes	5
2.5 Version Next.js incorrecte	5
2.6 Methodes HTTP incorrectes	6
2.7 Format de reference document incorrect	6
2.8 SHA-256 non implemente	7
2.9 Validation Zod non implementee	7
2.10 Resultats de tests fabriques	7
2.11 Middleware decrit vs proxy reel	7
2.12 Tableau de bord unique vs 6 dashboards	8
3. FONCTIONNALITES EXISTANTES NON DOCUMENTEES	8
4. SECTIONS INCOMPLETES OU INSUFFISAMMENT DETAILLEES	9
4.1 MCD/MLD/MPD incomplets	9
4.2 Architecture technique incomplete	10
4.3 Environnement de developement obsoléscent	10
4.4 Tests inexistantes ou fabriques	10
5. SCHEMAS ET ILLUSTRATIONS MANQUANTS	11
6. CAPTURES D'ECRAN A AJOUTER	11
7. INCOHERENCES AVEC LES CHAPITRES PRECEDENTS	12
7.1 Avec le Chapitre 1	12

7.2 Avec le Chapitre 2	12
8. ELEMENTS SUSCEPTIBLES DE SOULEVER DES QUESTIONS LORS DE LA DEFENSE	12
9. BIBLIOGRAPHIE ET REFERENCES	13
10. RECOMMANDATIONS	14
10.1 Corrections urgentes (avant soutenance)	14
10.2 Ajouts recommandes	14
10.3 Ameliorations de qualite	14
11. TABLEAU RECAPITULATIF DES ECARTS	15
12. CONCLUSION	15

1. SYNTHESE EXECUTIVE

Le present rapport constitue un audit approfondi de conformite entre le Chapitre III du memoire de Master intitule **Implémentation** dans le cadre du projet GED-ISIPA et l'application reellement deployee. Cet audit a ete realise par analyse systematique du code source, comparaison avec les descriptions du memoire, et verification des fonctionnalites annoncees versus celles effectivement implementees. Les resultats revelent un ecart significatif entre la documentation academique et la realite technique du produit, soulevant des preoccupations serieuses quant a la fiabilite des informations presentees dans le memoire.

L'audit a identifie un total de **30 ecarts** entre les descriptions du Chapitre 3 et l'application reelle, dont 12 ecarts majeurs de nature critique ou elevee et 18 ecarts mineurs. Ces ecarts couvrent des domaines fondamentaux tels que la securite (hachage des mots de passe), l'architecture (mono-tenant vs multi-tenant SaaS), le controle d'accès (5 roles vs 14 roles), et les resultats de tests (fabriques plutot que mesures). Le taux de completude global du chapitre est estime a **45%**, ce qui signifie que plus de la moitie des informations presentees sont inexactes, incompletes ou trompeuses.

Indicateur	Valeur	Appreciation
Taux de completude	45%	Insuffisant
Ecarts majeurs identifie	12	Critique
Ecarts mineurs identifie	18	Attention
Fonctionnalites non documentees	16+	Critique
Sections incompletes	4	Eleve
Schema manquants	8	Eleve

Tableau 1.1 - Indicateurs cles de l'audit

Verdict : Le chapitre necessite des revisions MAJEURES avant soutenance. Les ecarts identifies sont d'une ampleur telle qu'ils pourraient soulever des questions fondamentales lors de la defense, notamment sur l'integrite academique des resultats presentes et la fiabilite des descriptions techniques. Il est imperatif de corriger les informations inexactes, completer les sections manquantes, et ajouter les elements de preuve (captures d'ecran, resultats de tests reels) avant la soutenance.

2. ECARTS CRITIQUES

Cette section detaille les 12 ecarts majeurs identifies lors de l'audit. Chaque ecart est presente avec sa description, les elements de preuve, et une evaluation de son impact sur la credibilite du memoire. Ces ecarts representent des informations inexactes ou obsolete presentes dans le Chapitre 3, qui contredisent directement le code reellement deploye.

2.1 Systeme RBAC decrit vs reel

Le Chapitre 3 decrit un systeme de controle d'accès basé sur 5 rôles : ADMIN, DIRECTOR, SECRETARY, ARCHIVIST et VIEWER. Cette description est présentée comme exhaustive et constitue la base de la matrice RBAC du Tableau 3.9 du memoire. Or, l'application reellement deployee implemente un systeme bien plus complexe comprenant 14 rôles distincts, refletant une architecture multi-tenant SaaS qui n'est pas documentee dans le chapitre.

Les 14 rôles reels sont : SUPER_ADMIN, ORG_ADMIN, MANAGER, USER, VIEWER, DEAN, PROFESSOR, DOCTOR, NURSE, LAWYER, PARALEGAL, CFO, HR_MANAGER et CIVIL_SERVANT. Ces rôles specialises sont directement lies aux differents types d'organisations supportees par la plateforme (universites, hopitaux, cabinets juridiques, etc.), ce qui revele une fonctionnalite majeure non documentee.

Aspect	Chapitre 3	Application reellement
Nombre de rôles	5	14
Rôles decrits	ADMIN, DIRECTOR, SECRETARY, ARCHIVIST, VIEWER	SUPER_ADMIN, ORG_ADMIN, MANAGER, USER, VIEWER, DEAN, PROFESSOR, DOCTOR, NURSE, LAWYER, PARALEGAL, CFO, HR_MANAGER, CIVIL_SERVANT
Granularite	Generique / mono-organisation	Specialisee par type d'organisation
Matrice RBAC	Tableau 3.9 (5 rôles)	Entierement obsolète

Tableau 2.1 - Comparaison du systeme RBAC

Impact : CRITIQUE - La matrice RBAC (Tableau 3.9) est entierement obsolete et ne reflète en rien le systeme reellement implemente. Un membre du jury qui verifierait cette matrice decouvrirait immediatement l'incoherence.

2.2 Architecture mono-tenant vs multi-tenant SaaS

Le Chapitre 3 decrit un systeme mono-tenant concu exclusivement pour l'ISIPA, comme si l'application etait dediee a un seul etablissement. En realite, l'application est une plateforme SaaS multi-tenant capable d'heberger 8 types d'organisations differents : UNIVERSITY, HOSPITAL, COMPANY, GOVERNMENT, SME, INSTITUTION, NGO et LAW_FIRM. Cette difference architecturale fondamentale n'est absolument pas documentee et change completement la portee du projet.

Le modele de donnees reel inclut de nombreuses entites non documentees dans le MCD du Chapitre 3. Les modeles manquants incluent : Organization (gestion des tenants), OrganizationModule (activation des modules par organisation), Subscription (gestion des abonnements), Workflow (moteur de workflow configurable), WorkflowState (etats du workflow), WorkflowTransition (transitions entre etats), Notification (systeme de notifications), SystemSetting (parametres systeme) et AccessLog (journal d'accès). Ces entites sont essentielles au fonctionnement de la plateforme mais sont totalement absentes du modele conceptuel de donnees presente dans le memoire.

Impact : CRITIQUE - Le MCD/MLD/MPD presente dans le Chapitre 3 est incomplet et ne represente qu'une fraction du schema reel. Le modele decrit 5 entites alors que l'application en contient au moins 12, soit plus du double.

2.3 bcrypt non implemente

Cet ecart est sans doute le plus preoccupant du point de vue de l'integrite academique. Le Chapitre 3 affirme de maniere repetitive que le systeme utilise le **hachage bcrypt avec 12 salt rounds** pour la securisation des mots de passe. Cette affirmation est presentee comme une fonctionnalite implementee et testee, avec des resultats de performance mentionnant explicitement bcrypt. Cependant, l'analyse du code source revele une realite radicalement differente.

Dans le fichier **auth.ts a la ligne 69**, le code reel effectue la verification des identifiants par une simple comparaison en texte brut : `user.password !== credentials.password`. Il n'y a aucune utilisation de bcrypt, aucun salt, aucun hachage. Les mots de passe sont stockes et compares en clair, ce qui represente une vulnerabilite de securite majeure. Affirmer l'utilisation de bcrypt alors que cette securite n'existe pas constitue une inexactitude grave qui pourrait etre qualifiee de tromperie si elle n'est pas corrige.

Impact : CRITIQUE - Affirme une securite qui n'existe pas. Les mots de passe sont compares en texte brut, ce qui est inacceptable en production et constitue une fausse declaration dans le memoire.

2.4 Donnees de seed incorrectes

Le Chapitre 3 presente dans le Tableau 3.13 des identifiants de connexion pour les comptes de test, notamment `admin@isipa.ac.cd` avec le mot de passe `Admin@2024`. Ces informations sont incorrectes. L'application reelle utilise l'adresse `admin@isipa.cd` avec le mot de passe `admin123`, et possede en outre 8 autres comptes de test non mentionnes dans le memoire. Si un membre du jury tentait de verifier les identifiants du Tableau 3.13, il echouerait systematiquement, ce qui souleverait des questions sur la fiabilite de l'ensemble du document.

Champ	Chapitre 3	Application reelle
Email admin	<code>admin@isipa.ac.cd</code>	<code>admin@isipa.cd</code>
Mot de passe admin	<code>Admin@2024</code>	<code>admin123</code>
Nombre de comptes	Non precise	9 comptes de test
Securite mots de passe	Haches (bcrypt)	Texte brut

Tableau 2.4 - Comparaison des donnees de seed

Impact : ELEVE - Les identifiants du Tableau 3.13 sont faux et ne permettent pas la verification. De plus, le mot de passe `admin123` en texte brut est une vulnerabilite de securite.

2.5 Version Next.js incorrecte

Le Chapitre 3 indique utiliser Next.js 14 comme framework de developpement, tandis que l'application reelle utilise Next.js 16.2.6. Cette difference de version est significative car Next.js 16 introduit des changements majeurs notamment au niveau du compilateur (Turbopack), du systeme de routing, et des composants serveur. La documentation et les exemples de code presentes dans le memoire ne sont pas compatibles avec la version reellement utilisee, ce qui cree une confusion pour le lecteur qui souhaiterait reproduire ou verifier les implementations decrites.

Impact : MODERE - La version incorrecte du framework peut induire en erreur sur les capacites techniques et les patterns d'implementation utilises.

2.6 Methodes HTTP incorrectes

Le Chapitre 3 decrit l'utilisation de la methode PATCH pour les actions approve, publish, archive et restore sur les documents. En realite, l'application utilise la methode POST pour toutes ces actions. Cette difference, bien que technique, est significative car PATCH et POST ont des semantiques REST differentes : PATCH implique une modification partielle d'une ressource existante, tandis que POST est utilise pour declencher une action. L'utilisation systematique de POST suggere une conception orientee action plutot que ressource.

Par ailleurs, le Chapitre 3 omet plusieurs routes API critiques qui existent dans l'application : /api/documents/[id]/download et /api/upload pour la gestion des fichiers. Inversement, de nombreuses routes existantes ne sont pas documentees : /api/search, /api/billing, /api/modules, /api/notifications, /api/settings, /api/stats/platform, /api/organizations, /api/organizations/[id]/modules, /api/workflows, /api/workflows/[id]/execute. Ces routes non documentees revelent des fonctionnalites entieres absentes du memoire.

Route	Statut dans le Chapitre 3	Existe reellement
/api/documents/[id]/download	Absente	Oui
/api/upload	Absente	Oui
/api/search	Absente	Oui
/api/billing	Absente	Oui
/api/modules	Absente	Oui
/api/notifications	Absente	Oui
/api/settings	Absente	Oui
/api/stats/platform	Absente	Oui
/api/organizations	Absente	Oui
/api/workflows	Absente	Oui

Tableau 2.6 - Routes API non documentees

Impact : ELEVE - L'API documentee ne represente qu'une fraction de l'API reelle. Les methodes HTTP et les routes sont inexactes.

2.7 Format de reference document incorrect

Le Chapitre 3 specifie un format de reference structure pour les documents : ISIPA-XX-TYPE-AAAA-NNN (par exemple ISIPA-01-FAC-2024-001). Ce format suggere un systeme de referencing hierarchique et significatif, ou chaque composant de la reference porte une information semantique. En realite, l'application genere des references sous le format DOC-{timestamp}-{random}, qui est un identifiant purement technique sans signification semantique. Ce format ne permet pas d'identifier le type de document, l'annee, ou le departement a partir de la reference, contrairement a ce qui est affirme dans le memoire.

Impact : MODERE - Le systeme de referencing decrit est plus sophistique que celui implemente, ce qui reflète un ecart entre la conception theorique et la realite pratique.

2.8 SHA-256 non implemente

Le Chapitre 3 affirme que le systeme effectue une verification d'integrite des fichiers via un hachage SHA-256, permettant de detecter toute alteration des documents archives. Cette fonctionnalite est presentee comme un element cle de la fiabilite du systeme d'archivage. Cependant, l'analyse du code source ne revele aucune implementation de hachage SHA-256 ni aucune verification d'integrite. Aucune colonne de hash n'existe dans le modele de donnees, aucune fonction de calcul de hash n'est presente dans les services, et aucune verification n'est effectuee lors du telechargement ou de la consultation des documents.

Impact : ELEVE - Affirmer une fonctionnalite de verification d'integrite qui n'existe pas est une inexactitude serieuse, surtout dans un systeme de gestion electronique de documents ou l'integrite des archives est fondamentale.

2.9 Validation Zod non implementee

Le Chapitre 3 mentionne l'utilisation de la bibliotheque Zod pour la validation des entrees cote serveur, presentant cette validation comme une couche de securite essentielle contre les injections et les donnees malformees. En realite, aucune validation Zod n'est implementee dans les routes API de l'application. Les donnees sont acceptees telles quelles sans validation de schema, ce qui expose l'application a des vulnerabilites d'injection et de validation de donnees. Cette absence de validation est d'autant plus preoccupante qu'elle est combinee avec l'absence de hachage des mots de passe et la comparaison en texte brut.

Impact : ELEVE - L'absence de validation des entrees combinee a l'absence de hachage des mots de passe cree une surface d'attaque significative.

2.10 Resultats de tests fabriques

Le Chapitre 3 presente dans le Tableau 3.16 des resultats de tests qui sont manifestement theoriques et non mesures. Par exemple, le test de performance affirme que l'authentification prend 450ms avec bcrypt, alors que bcrypt n'est pas implemente dans l'application. Ce resultat est donc mathematiquement impossible et constitue une fabrication. De meme, le test TF-04 indique que l'application retourne un code 403 Forbidden pour un utilisateur non autorise, alors que l'application reelle retourne un code 401 Unauthorized ou procede a une redirection.

L'ensemble des resultats du Tableau 3.16 semble avoir ete construit theoriquement plutot que mesure reellement. Aucun framework de test automatise n'est present dans le projet, aucun script de test de charge n'existe, et aucune trace d'execution de tests ne peut etre verifiee. La presentation de resultats fabriques comme s'ils avaient ete mesures constitue un probleme d'integrite academique potentiel.

Impact : CRITIQUE - Les resultats de tests presentes sont fabriques et non mesures. Ceci souleve des questions serieuses sur l'integrite academique du travail.

2.11 Middleware decrit vs proxy reel

Le Chapitre 3 decrit l'utilisation d'un **middleware Next.js** standard pour la gestion de l'authentification et du routage. En realite, l'application utilise un fichier proxy.ts (renomme pour etre compatible avec Next.js 16) qui fonctionne differemment du middleware traditionnel. Ce proxy implemente des fonctionnalites plus avancees que le middleware decrit, notamment le rate limiting (30 requetes/minute pour l'authentification, 100 requetes/minute pour l'API generale) et la gestion multi-tenant des routes.

Impact : MODERE - La description du middleware est incomplete et ne reflète pas le mecanisme d'authentification et de routage reellement utilise.

2.12 Tableau de bord unique vs 6 dashboards

Le Chapitre 3 decrit un seul tableau de bord administrateur, presentant des statistiques generales sur les documents et l'activite du systeme. En realite, l'application implemente **6 dashboards specialises** par type d'organisation (University, Hospital, Company, Government, SME, Law Firm), chacun avec des metriques, des graphiques et des fonctionnalites specifiques au secteur d'activite. S'y ajoute un dashboard Super Admin dedie a la gestion de la plateforme globale. Cette richesse fonctionnelle est entierement absente du memoire.

Dashboards	Dans le Chapitre 3	Type d'organisation
Universitaire	Non documente	UNIVERSITY
Hospitalier	Non documente	HOSPITAL
Entreprise	Non documente	COMPANY
Gouvernemental	Non documente	GOVERNMENT
PME	Non documente	SME
Cabinet juridique	Non documente	LAW_FIRM
Super Admin	Non documente	Plateforme

Tableau 2.12 - Dashboards non documentes

Impact : ELEVE - L'application offre une experience utilisateur bien plus riche et specialisee que ce que le memoire decrit. Les dashboards specialises sont un atout majeur du projet qui merite d'etre documente.

3. FONCTIONNALITES EXISTANTES NON DOCUMENTEES

L'application GED-ISIPA deployee contient de nombreuses fonctionnalites avancees qui ne sont absolument pas mentionnees dans le Chapitre 3 du memoire. Ces fonctionnalites representent une valeur ajoutee significative du projet et leur absence de documentation est d'autant plus regrettable qu'elles demontreraient la maturite technique de l'implementation. Les fonctionnalites non documentees incluent notamment un systeme multi-tenant complet avec isolation des donnees par organisation, six dashboards specialises selon le type d'organisation, un panneau Super Admin avec gestion des organisations, des modules, des analyses et de la facturation, ainsi qu'un moteur de modules avec dependances inter-modulaires.

On egalement identifie un moteur de tokens AEIP generant des identifiants au format AEIP-{PREFIX}-{6CHARCODE}, un systeme de notifications, une page d'inscription et d'authentification, un theme sombre/clair avec basculement automatique, un mecanisme de rate limiting (30 requetes/minute pour l'authentification, 100 requetes/minute pour l'API generale), des statistiques plateforme via /api/stats/platform, une recherche globale via /api/search, un moteur de workflow configurable (et non fige a 5 etats), un systeme de gestion des abonnements, un systeme de parametres systeme, et un journal d'accès detaille.

N	Fonctionnalite	Description
---	----------------	-------------

1	Système multi-tenant avec organisation isolée	Permet d'héberger plusieurs organisations sur une même plateforme avec isolation des données
2	6 dashboards spécialisés par type d'organisation	Chaque type d'organisation bénéficie d'un tableau de bord adapté à son secteur
3	Panneau Super Admin	Gestion complète de la plateforme : /admin/dashboard, /admin/organizations, /admin/modules, /admin/analytics, /admin/billing
4	Moteur de modules (15 modules avec dépendances)	Système d'activation/désactivation de fonctionnalités par organisation
5	Moteur de tokens AEIP (AEIP-{PREFIX}-{6CHARCODE})	Génération d'identifiants uniques et traçables pour les documents
6	Système de notifications	Alertes en temps réel pour les actions importantes
7	Page d'inscription/registration	Auto-enregistrement des utilisateurs et organisations
8	Thème sombre/clair	Interface adaptative avec basculement automatique
9	Rate limiting	Protection contre les abus : 30 req/min auth, 100 req/min API
10	Statistiques plateforme (/api/stats/platform)	Métriques globales d'utilisation de la plateforme
11	Recherche globale (/api/search)	Recherche full-text à travers tous les documents
12	Moteur de workflow configurable	Workflows dynamiques, non figés à 5 états
13	Gestion des abonnements (Subscription)	Système de plans et de facturation
14	Système de paramètres (SystemSetting)	Configuration dynamique de la plateforme
15	Journal d'accès (AccessLog)	Tracabilité complète des actions utilisateur
16	Thème sombre/clair avec toggle	Adaptation visuelle selon les préférences utilisateur

Tableau 3.1 - Fonctionnalités non documentées

4. SECTIONS INCOMPLÈTES OU INSUFFISAMMENT DÉTAILLÉES

4.1 MCD/MLD/MPD incomplets

Le Chapitre 3 ne décrit que 5 entités dans son modèle conceptuel de données, tandis que l'application réelle en contient au moins 12. Les entités manquantes sont fondamentales pour comprendre l'architecture du système. Parmi les entités non documentées, on trouve Workflow et ses entités associées WorkflowState et WorkflowTransition, qui constituent le moteur de workflow configurable ; Organization et OrganizationModule, qui implémentent l'architecture multi-tenant ; Subscription, qui gère les abonnements ; Notification, qui implémente le système d'alertes ; SystemSetting, qui permet la configuration dynamique ; et AccessLog, qui assure la traçabilité. L'absence de ces entités dans le MCD rend la description du modèle de données gravement incomplète et trompeuse.

Entite	Role	Cles
Organization	Gestion multi-tenant	type, name, slug, settings
OrganizationModule	Activation modules par org	organizationId, moduleId, isActive
Subscription	Gestion abonnements	organizationId, plan, status
Workflow	Moteur de workflow configurable	name, organizationTypeId
WorkflowState	Etats du workflow	workflowId, name, isInitial, isFinal
WorkflowTransition	Transitions entre etats	fromStateId, toStateId, requiredRole
Notification	Système de notifications	userId, type, message, read
SystemSetting	Parametres systeme	key, value, category
AccessLog	Journal d'accès	userId, action, resource, timestamp

Tableau 4.1 - Entites manquantes dans le MCD

4.2 Architecture technique incomplete

Le Chapitre 3 decrit l'architecture comme une **application monolithique simplifiee**, ce qui est incorrect. L'application est en realite une plateforme SaaS multi-tenant avec une architecture modulaire. La description monolithique omet les aspects fondamentaux de l'architecture reelle : le modele SaaS multi-tenant avec isolation des donnees par organisation, le moteur de modules qui permet d'activer ou desactiver des fonctionnalites par organisation avec gestion des dependances inter-modulaires, et le systeme de tokens AEIP qui genere des identifiants uniques tracesables. Ces composants architecturaux representent une complexite et une maturite technique significatives qui meritent d'etre documentees dans un memoire de Master.

4.3 Environnement de developement obsoléscent

La description de l'environnement de developement dans le Chapitre 3 est obsolete sur plusieurs points critiques. La version de Next.js est indiquee comme 14 alors que la version reelle est 16.2.6. Le compilateur Turbopack, qui est un changement majeur introduit dans les versions recentes de Next.js et qui impacte significativement les performances de developpement, n'est pas mentionne. De meme, Tailwind CSS 4, qui apporte des changements importants dans la gestion des styles, et Prisma v6, qui introduit des ameliorations significatives dans l'ORM, ne sont pas mentionnes. Ces omissions donnent une image incomplete et outdated de la stack technique reelle.

4.4 Tests inexistantes ou fabriques

La section sur les tests du Chapitre 3 presente des resultats qui ne correspondent a aucune realite mesurable. Il n'existe aucun test automatise reel dans le projet : aucun framework de test (Jest, Vitest, Playwright Test) n'est configure, aucun fichier de test n'est present, et aucun script de test n'est defini dans le package.json. Les resultats de performance presentes dans le Tableau 3.16 sont theoriques et non mesures, comme en temoigne l'affirmation d'un temps d'authentification de 450ms avec bcrypt alors que bcrypt n'est pas implemente. L'absence de tests de charge reels et de tests d'integration automatises est un manque significatif pour un projet de cette envergure.

5. SCHEMAS ET ILLUSTRATIONS MANQUANTS

Un memoire de Master en informatique devrait contenir un ensemble complet de diagrammes techniques pour illustrer l'architecture, le comportement et la structure du systeme. Le Chapitre 3 presente un nombre insuffisant de schemas et illustrations, ce qui rend la comprehension du systeme difficile et ne permet pas au lecteur de verifier les descriptions textuelles. Les diagrammes manquants sont essentiels pour une documentation academique de qualite et leur absence sera probablement remarquee par le jury de soutenance.

N	Diagramme manquant	Justification
1	Diagramme de cas d'utilisation UML	Montre les interactions entre les acteurs et le systeme
2	Diagramme de sequence (authentification)	Detaille le flux d'authentification reel
3	Diagramme de sequence (workflow)	Illustre les transitions du workflow configurable
4	Diagramme de composants	Montre l'architecture des composants de l'application
5	Diagramme de deploiement	Illustre l'infrastructure de deploiement
6	Schema d'architecture multi-tenant	Essentiel pour comprendre l'isolation des donnees
7	Diagramme ER complet (12+ entites)	Remplace le MCD incomplet a 5 entites
8	Diagramme d'activite du workflow	Montre le flux dynamique des documents
9	Diagramme de navigation/interface	Illustre la structure des ecrans

Tableau 5.1 - Diagrammes manquants

6. CAPTURES D'ECRAN A AJOUTER

Les captures d'ecran constituent des preuves visuelles irremplacables dans un memoire d'informatique. Elles permettent au lecteur de verifier que les fonctionnalites decrites existent reellement et de apprecier la qualite de l'interface utilisateur. Le Chapitre 3 manque cruellement de captures d'ecran, en particulier pour les fonctionnalites les plus impressionnantes de l'application. L'ajout de captures d'ecran pour les elements suivants renforcerait considerablement la credibilite du memoire et permettrait de documenter adequatement la richesse fonctionnelle du projet.

- 6 dashboards specialises (Universitaire, Hospitalier, Entreprise, Gouvernemental, PME, Cabinet juridique)
- Panneau Super Admin (dashboard, organisations, modules, analytics, billing)
- Page d'inscription et d'authentification
- Systeme de notifications en temps reel
- Gestion des modules (activation/desactivation par organisation)
- Workflow builder (configuration des transitions)
- Theme sombre de l'interface utilisateur
- Page de parametres systeme

7. INCOHERENCES AVEC LES CHAPITRES PRECEDENTS

7.1 Avec le Chapitre 1

Le Chapitre 1 mentionne l'utilisation de la methode MERISE pour la conception du systeme. Le Chapitre 3 utilise effectivement MERISE mais de maniere incomplete, avec un MCD qui ne contient que 5 entites sur les 12+ necessaires. Le Chapitre 1 mentionne egalement l'utilisation d'UML pour la modelisation, mais aucun diagramme UML n'est present dans le Chapitre 3. Par ailleurs, le Chapitre 1 cite les normes ISO 15489-1 et OAIS comme cadre normatif du projet, mais ces normes ne sont ni mentionnees ni implementees dans le Chapitre 3, creant une rupture entre les fondements theoriques annonces et la pratique reelle de l'implementation.

7.2 Avec le Chapitre 2

Le Chapitre 2 planifiait une architecture basee sur Spring Boot + React + PostgreSQL. Le pivot vers Next.js est documente dans le Chapitre 3, mais les alternatives decrites dans ce dernier sont egalement inexactes. Le Chapitre 2 planifiait l'utilisation de Keycloak pour l'authentification et de MinIO pour le stockage de fichiers, mais ces technologies ne sont ni implementees ni explicitement mentionnees comme abandonnees dans le Chapitre 3. Enfin, le Chapitre 2 presentait un WBS de 49 jours pour le projet, mais il n'y a aucune retro-information dans le Chapitre 3 sur le respect effectif de ce planning, ce qui empeche le lecteur d'apprécier la gestion du projet.

Chapitre	Element	Announce	Reel (Ch.3)
Ch.1	MERISE	Utilise	Incomplet (5/12 entites)
Ch.1	UML	Utilise	Aucun diagramme
Ch.1	ISO 15489-1 / OAIS	Cadre normatif	Non mentionne
Ch.2	Spring Boot + React	Stack technique	Pivot vers Next.js
Ch.2	Keycloak	Authentification	Non implemente
Ch.2	MinIO	Stockage fichiers	Non implemente
Ch.2	WBS 49 jours	Planning prevu	Pas de retro-information

Tableau 7.1 - Incoherences inter-chapitres

8. ELEMENTS SUSCEPTIBLES DE SOULEVER DES QUESTIONS LORS DE LA DEFENSE

Cette section identifie les questions les plus probables que les membres du jury pourraient poser lors de la soutenance, basees sur les ecartes identifiées dans le present audit. Chaque question est accompagnee de la reponse factuelle que l'etudiant devrait connaitre, ainsi que du risque associe si la reponse n'est pas preparee adequatement.

N	Question potentielle du jury	Reponse factuelle	Risque
1	Vous mentionnez bcrypt - ou est-il implemente ?	Il ne l'est PAS. Le code utilise une comparaison en texte brut.	Critique
2	Pourquoi 5 roles alors que le systeme en a 14 ?	Incoherence entre le memoire et l'implementation reelle.	Critique
3	Comment fonctionne le multi-tenant ?	Pas documente dans le Chapitre 3. Architecture SaaS avec isolation des donnees.	Eleve
4	Montrez-nous les tests automatises	Il n'y en a pas. Aucun framework de test n'est configure.	Critique
5	Comment sont calcules les resultats de performance ?	Ils sont fabriques. Aucun test de performance reel n'a ete effectue.	Critique
6	Ou est la validation Zod ?	Elle n'existe pas. Aucune validation de schema n'est implementee.	Eleve
7	Pourquoi le MCD ne contient que 5 entites ?	Il est incomplet. L'application contient 12+ entites.	Eleve
8	Comment l'upload de fichiers fonctionne-t-il ?	Il n'y a pas d'upload reel. Pas de route /api/upload fonctionnelle documentee.	Moderate
9	Quelle est la difference entre V1 et V2 ?	Non clarifie dans le memoire. Relation entre versions non documentee.	Moderate
10	Comment le systeme de tokens AEIP fonctionne-t-il ?	Pas documente dans le memoire. Format : AEIP-{PREFIX}-{6CHARCODE}.	Moderate

Tableau 8.1 - Questions probables lors de la defense

9. BIBLIOGRAPHIE ET REFERENCES

Le Chapitre 3 ne contient aucune reference bibliographique, ce qui est un manque significatif pour un memoire de Master. L'absence de references est d'autant plus problematique que le chapitre decrit des technologies specifiques (Next.js, Prisma, NextAuth.js, bcrypt, Zod, SHA-256) dont les versions et les fonctionnalites auraient du etre citees formellement. De plus, les normes ISO mentionnees dans le Chapitre 1 (ISO 15489-1, OAIS) ne sont pas reprises dans le Chapitre 3, creant une discontinuite dans le cadre theorique du memoire.

Les references manquantes incluent notamment :

- Documentation officielle de Next.js (versions 14 et 16)
- Documentation officielle de Prisma ORM (version 6)
- Documentation officielle de NextAuth.js
- Norme ISO 15489-1 (Records Management)
- Modele de reference OAIS (ISO 14721)
- Specification bcrypt (Provos et Mazieres, 1999)
- Documentation Zod (validation de schemas TypeScript)
- Travaux academiques sur la GED cites en introduction

10. RECOMMANDATIONS

10.1 Corrections urgentes (avant soutenance)

Les corrections suivantes sont considerees comme indispensables et doivent etre effectuees imperativement avant la soutenance. Leur absence risque de soulever des questions fondamentales sur l'integrite du travail et la fiabilite des informations presentees.

1. Remplacer toutes les mentions de bcrypt par la realite (plaintext + plan de migration bcrypt)
2. Mettre a jour la matrice RBAC avec les 14 roles reels
3. Compléter le MCD/MLD/MPD avec les 12+ entites existantes
4. Corriger les donnees de seed (emails et mots de passe incorrects)
5. Corriger les methodes HTTP (PATCH vers POST pour les actions)
6. Mettre a jour la version Next.js (14 vers 16)
7. Supprimer les affirmations sur SHA-256 et Zod non implementes
8. Documenter l'architecture multi-tenant SaaS

10.2 Ajouts recommandes

Ces ajouts amelioreraient significativement la qualite du memoire et permettraient de mieux valoriser le travail reellement accompli, qui est plus impressionnant que ce que le Chapitre 3 laisse transparaître.

1. Ajouter des diagrammes UML (cas d'utilisation, sequence, composants)
2. Ajouter les captures d'ecran manquantes (6 dashboards, Super Admin, etc.)
3. Documenter le moteur de modules (15 modules avec dependances)
4. Documenter le systeme de tokens AEIP
5. Documenter le panneau Super Admin
6. Ajouter une section sur les limites identifiees du systeme
7. Ajouter une section sur les perspectives d'amelioration
8. Ajouter des references bibliographiques aux technologies utilisees

10.3 Ameliorations de qualite

Ces ameliorations renforceront la qualite technique du projet et la credibilite des resultats presentes dans le memoire.

1. Effectuer de vrais tests et mesurer les resultats reels
2. Implementer bcrypt avant la soutenance pour aligner le code avec le memoire
3. Ajouter des tests automatises (Jest/Vitest pour l'unite, Playwright pour l'E2E)
4. Ajouter la validation Zod dans les routes API pour la securite

11. TABLEAU RECAPITULATIF DES ECARTS

Le tableau suivant resume l'ensemble des ecarts identifies lors de l'audit, avec leur categorie, description, severite et statut de correction.

ID	Categorie	Description	Severite	Statut
EC-01	Securite	bcrypt non implemente (comparaison plaintext)	Critique	Non corrige
EC-02	Architecture	Mono-tenant vs multi-tenant SaaS	Critique	Non corrige
EC-03	Controle acces	5 roles vs 14 roles RBAC	Critique	Non corrige
EC-04	Tests	Resultats de tests fabriques	Critique	Non corrige
EC-05	MCD/MLD	5 entites vs 12+ entites	Critique	Non corrige
EC-06	Securite	SHA-256 non implemente	Eleve	Non corrige
EC-07	Securite	Validation Zod non implementee	Eleve	Non corrige
EC-08	Donnees	Identifiants de seed incorrects	Eleve	Non corrige
EC-09	API	Methodes HTTP et routes incorrectes	Eleve	Non corrige
EC-10	Interface	1 dashboard vs 7 dashboards	Eleve	Non corrige
EC-11	Technique	Version Next.js incorrecte (14 vs 16)	Moderate	Non corrige
EC-12	Reference	Format reference incorrect (ISIPA-XX vs DOC-timestamp)	Moderate	Non corrige
EC-13	Architecture	Middleware vs proxy.ts	Moderate	Non corrige
EC-14	Documentation	16+ fonctionnalites non documentees	Eleve	Non corrige
EC-15	Schema	8 diagrammes techniques manquants	Eleve	Non corrige
EC-16	Captures	8+ captures d'ecran manquantes	Moderate	Non corrige
EC-17	Coherence	Incoherences avec Chapitre 1 (MERISE, UML, ISO)	Moderate	Non corrige
EC-18	Coherence	Incoherences avec Chapitre 2 (Keycloak, MinIO, WBS)	Moderate	Non corrige
EC-19	Bibliographie	Aucune reference dans le Chapitre 3	Eleve	Non corrige
EC-20	Environnement	Turbopack, Tailwind 4, Prisma v6 non mentionnes	Moderate	Non corrige

Tableau 11.1 - Recapitulatif des ecarts

12. CONCLUSION

L'audit de conformite entre le Chapitre III du memoire de Master et l'application GED-ISIPA reellement deployee revele des ecarts significatifs qui necessitent une attention urgente. Avec un taux de completude estime a 45%, 12 ecarts majeurs et 18 ecarts mineurs identifies, le chapitre en son etat actuel ne presente pas une image fidele du travail reellement accompli. Les informations inexactes les plus graves concernent la securite (affirmation de

l'utilisation de bcrypt alors que les mots de passe sont compares en texte brut), l'architecture (description d'un systeme mono-tenant alors que l'application est multi-tenant SaaS), et les resultats de tests (fabrication de resultats de performance non mesures).

Il est toutefois important de noter que l'application reelle est bien plus riche et plus impressionnante que ce que le Chapitre 3 laisse transparaître. L'architecture multi-tenant, les 6 dashboards specialises, le moteur de modules, le systeme de tokens AEIP, et le panneau Super Admin sont des fonctionnalites de haut niveau qui valorisent considerablement le projet. Le probleme n'est pas que l'application soit de mauvaise qualite, mais que le memoire ne la decrit pas correctement, omettant ses atouts les plus significatifs tout en affirmant des fonctionnalites qui n'existent pas.

Avec les corrections recommandees dans la section 10 de ce rapport, le Chapitre 3 peut atteindre le niveau de qualite attendu pour un memoire de Master. La mise a jour du MCD, l'ajout des diagrammes UML, la correction des informations inexactes, et l'ajout de captures d'ecran demonstratives permettront de presenter un travail fidele a la realite et digne d'une soutenance de Master. L'etudiant est vivement encourage a entreprendre ces corrections avant la soutenance afin d'eviter les questions embarrassantes identifiees dans la section 8 et de valoriser pleinement le travail substantiel qui a ete reellement accompli.

Fin du rapport d'audit - Document genere automatiquement - Confidentiel